

USB Docking System Driver Guide

This guide describes the current WinUSB-based installation path for the CDAS USB docking device. It replaces the older WDK 7600, Windows 7 cross-certificate, and custom kernel-driver signing flow.

The active driver package uses the Windows in-box winusb.sys driver. The repository does not build or install the old cvisionusb.sys / cvusb64.sys custom kernel driver.

Supported Windows Versions

This package is intended for:

- Windows 8
- Windows 10
- Windows 11

Windows XP, Windows Vista, and Windows 7 are not supported by the active WinUSB package. Historical files are kept under Old_files for reference only.

Files Used

```
src_new\sys\cdas_winusb.inf
src_new\exe_wtih_wrapper\capsousb_test.vcxproj
src_new\exe_no_wrapper\capsousb_test_no_wrapper.vcxproj
dist\capsousb_test_with_wrapper.exe
dist\capsousb_test_no_wrapper.exe
```

The INF binds matching CDAS USB devices to winusb.sys. The sample applications communicate with the device through WinUSB from user mode.

Hardware IDs

The current INF includes these USB hardware IDs:

```
USB\VID_03EB&PID_941C
USB\VID_0638&PID_0931
USB\VID_03EB&PID_952C
```

Before deployment, confirm the actual device VID/PID in Device Manager or usbview.exe. If the device has a different VID/PID, update src_new\sys\cdas_winusb.inf before packaging.

Verify the INF

Use the WDK InfVerif.exe tool:

```
"C:\Program Files (x86)\Windows Kits\10\Tools\10.0.26100.0\x64\infverif.exe" /v src_new\sys\cdas_winusb.inf
```

Expected result:

```
INF is VALID
```

Signing Requirement

The repository provides an INF template but does not include a generated catalog file or private signing key.

For production deployment, generate the catalog and sign the driver package according to the target Windows security policy. Windows 10 and Windows 11 production systems normally require a trusted signed driver package.

For engineering-only testing, Windows test-signing mode may be used according to the organization's driver test policy.

Do not use the old Windows 7 GlobalSign cross-certificate procedure for this WinUSB package.

Install the Driver

Open an elevated Command Prompt or PowerShell window from the repository root.

Install the INF:

```
pnputil /add-driver src_new\sys\cdas_winusb.inf /install
```

If Windows does not immediately bind the device to the new driver, unplug and reconnect the CDAS USB docking device.

Verify Driver Binding

In Device Manager:

- Find the CDAS USB device.
- Open device properties.
- Open driver details.
- Confirm that `winusb.sys` is listed.

You can also verify the device VID/PID with `usbview.exe`.

Build the Sample Applications

The sample applications have been verified with VS2022 Build Tools.

```
call "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat" x86
MSBuild.exe src_new\exe_wtih_wrapper\capsousb_test.vcxproj /p:Configuration=Debug /p:Platform=Win32 /t:Build
MSBuild.exe src_new\exe_no_wrapper\capsousb_test_no_wrapper.vcxproj /p:Configuration=Debug /p:Platform=Win32 /
```

Build outputs:

```
src_new\exe_wtih_wrapper\Debug\capsousb_test_with_wrapper.exe
src_new\exe_no_wrapper\Debug\capsousb_test_no_wrapper.exe
```

The repository also keeps prebuilt debug executables:

```
dist\capsousb_test_with_wrapper.exe
dist\capsousb_test_no_wrapper.exe
```

Run the Sample Applications

After the device is bound to `winusb.sys`, run one of the samples:

```
dist\capsousb_test_with_wrapper.exe  
dist\capsousb_test_no_wrapper.exe
```

The with-wrapper sample keeps the old BulkUsb-style sample call names and maps them to WinUSB in `src_new\include\winusb_compat.h` and `src_new\lib\winusb_compat.cpp`.

The no-wrapper sample calls SetupAPI and WinUSB directly.

Troubleshooting

the target machine policy.

WinUSB package again.

interface GUID, and Device Manager driver details.

bulk-out endpoints.

endpoint selection in the sample transport code.

- If `pnputil` fails, confirm that the INF has a matching signed catalog for
- If Windows binds another driver, remove the old binding and install the
- If the sample reports that the device cannot be found, verify the VID/PID,
- If transfers fail, confirm that the device exposes the expected bulk-in and
- If the device exposes multiple endpoints in the same direction, add explicit

Rollback

To remove the installed package:

```
pnputil /enum-drivers  
pnputil /delete-driver oemXX.inf /uninstall
```

Replace `oemXX.inf` with the published INF name reported by `pnputil`.